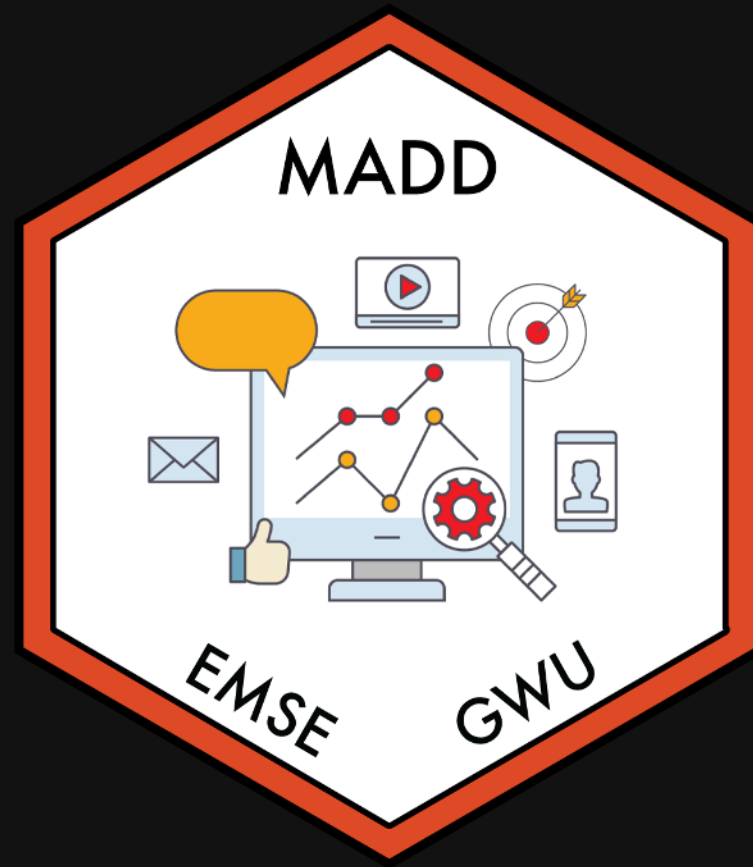




Intro to surveydown

🏛️ EMSE 6035: Marketing Analytics for Design Decisions

👤 John Paul Helveston

📅 September 30, 2026

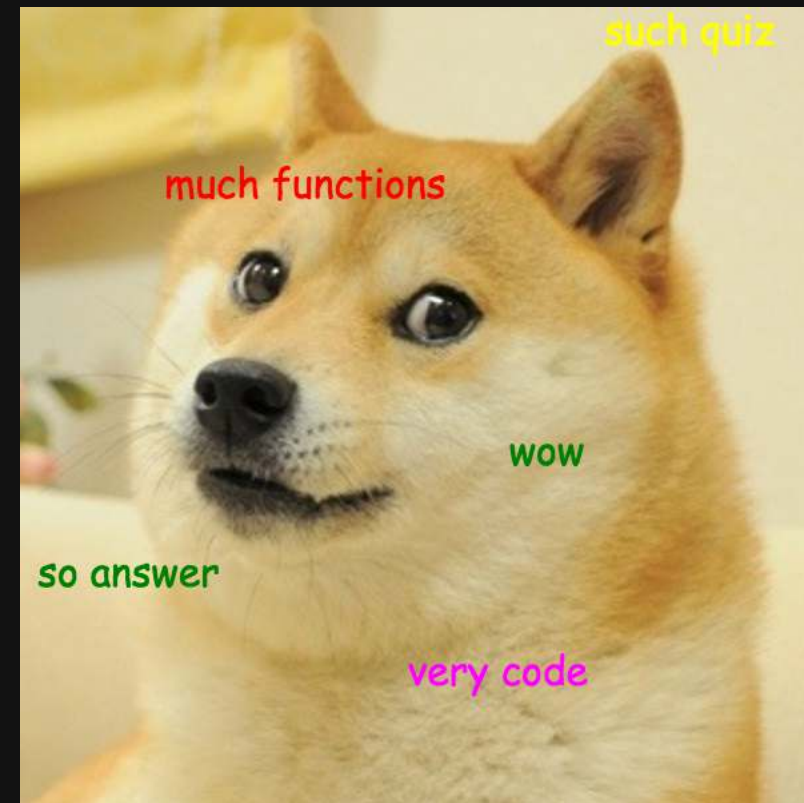
Quiz 2

10:00

Write your name on the quiz!

Rules:

- Work alone; no outside help of any kind is allowed.
- No calculators, no notes, no books, no computers, no phones.



Week 6: *Intro to surveydown*

1. What is surveydown?
2. Survey basics
3. Conditional logic
4. Storing & fetching data
5. Reactivity

Before we start...

1. Install surveydown

```
1 install.packages("surveydown")
```

2. Download today's class files

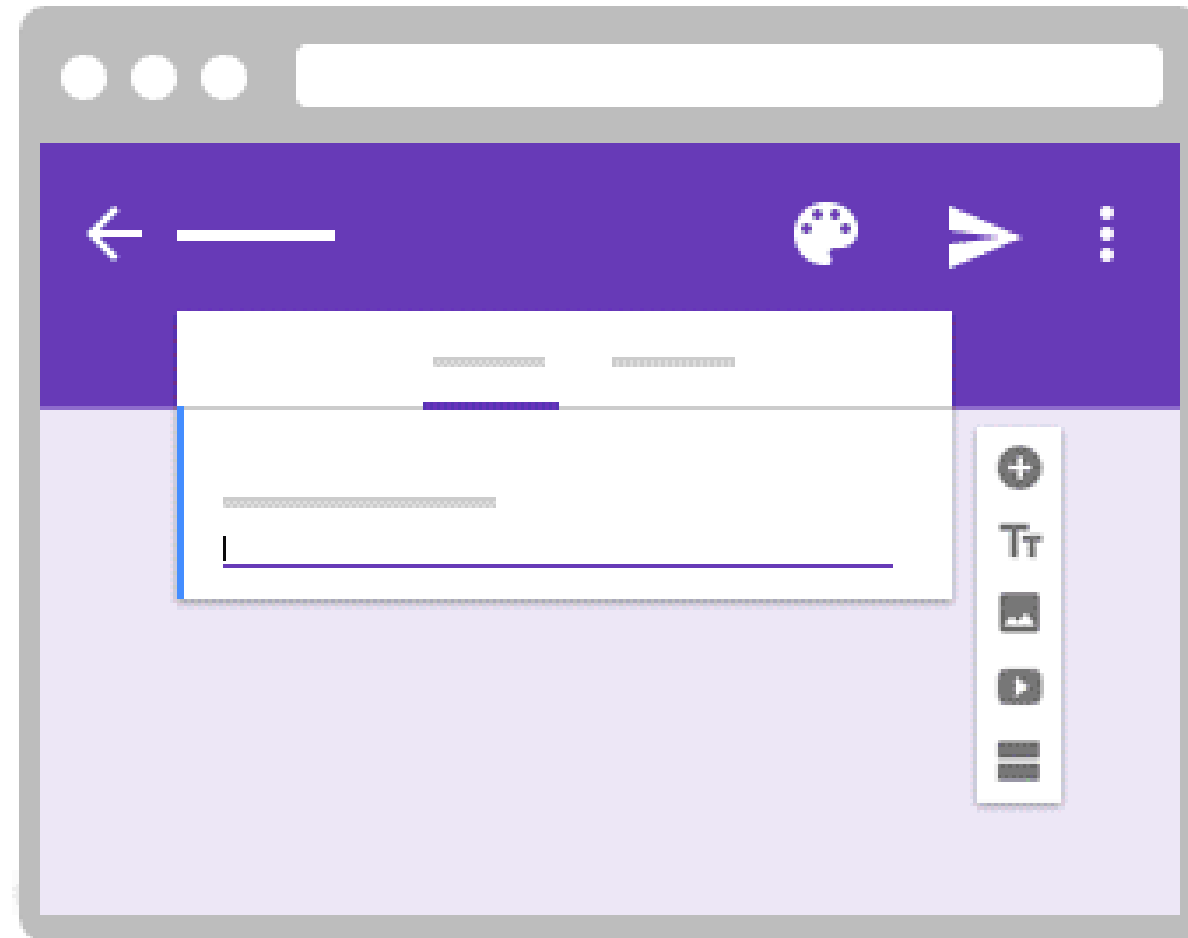
Each `demo-*` folder is a complete survey.

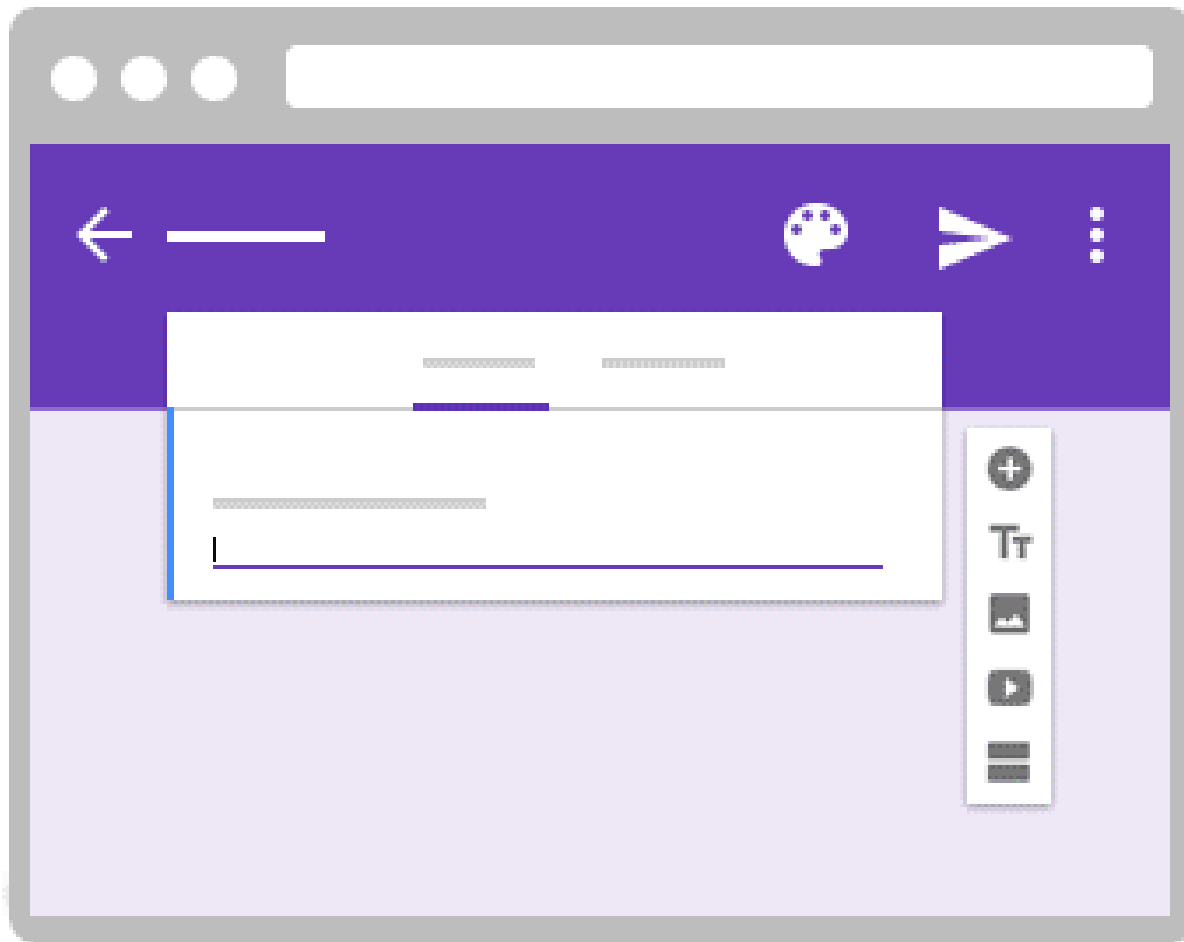
Open one in Positron with **File › Open Folder...**

Week 6: *Intro to surveydown*

1. What is surveydown?
2. Survey basics
3. Conditional logic
4. Storing & fetching data
5. Reactivity

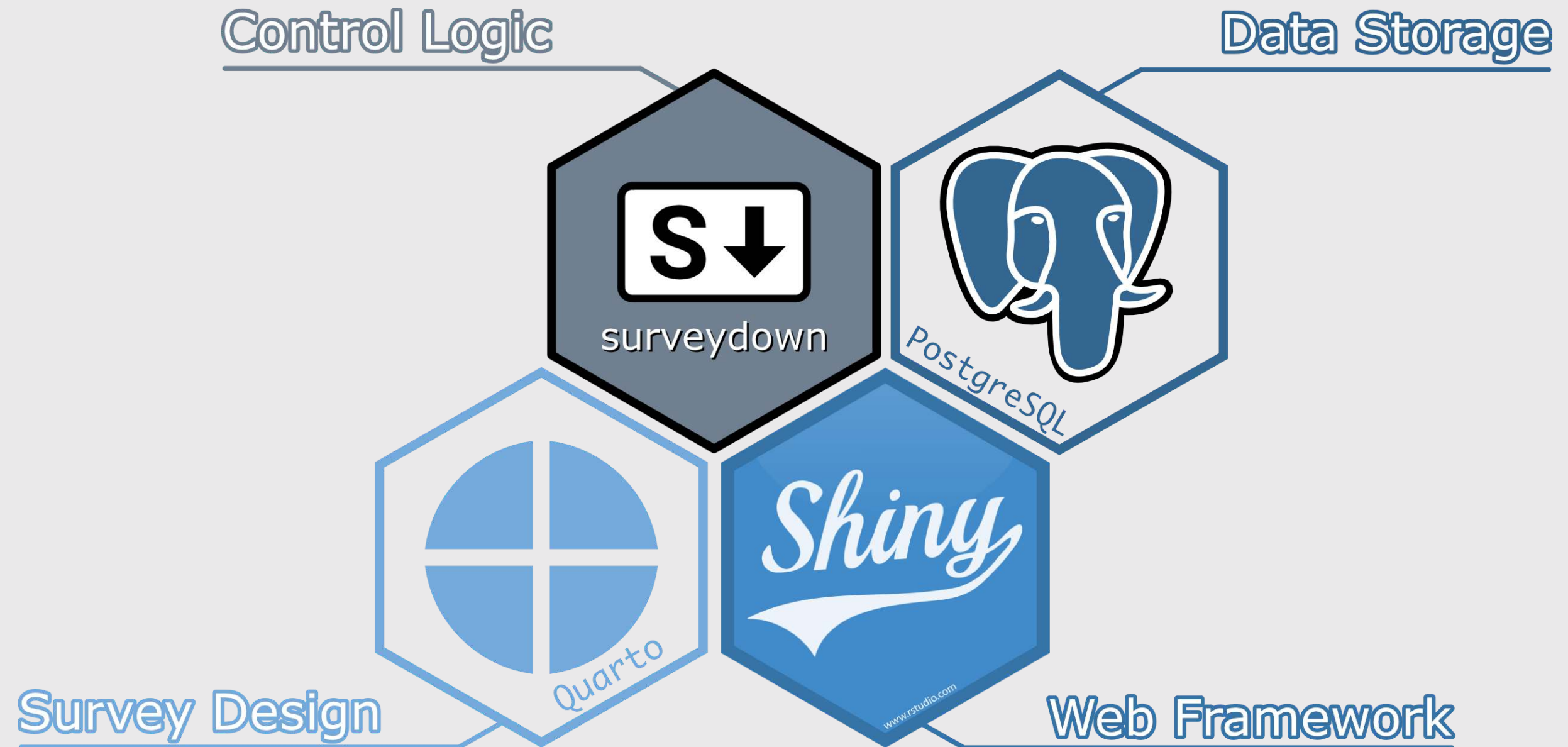
Typical experience making a survey





- ✗ Reproducible
- ✗ Data Control
- ⚠ Free to Use
- Open Source
- Easy Collaboration
- ⚠ Feature Packed

Introducing **surveydown**



Components

`survey.qmd`

A **Quarto doc** defining the main survey content (pages, text, images, questions, etc).

`app.R`

An **R script** defining the survey Shiny app. It sets up the database and server, and launches the survey.

survey.qmd

```
1 ---
2 survey-settings:
3   mode: preview
4 ---
5
6 ```{r}
7 library(surveydown)
8 ```
9
10 --- welcome
11
12 # Welcome to our survey!
13
14 ```{r}
15 sd_question(
16   type = "mc",
17   id = "penguins",
18   label = "What's your favorite penguin?",
19   option = c(
20     "Adélie" = "adelie",
21     "Chinstrap" = "chinstrap",
22     "Gentoo" = "gentoo"
23   )
24 )
25 ```
26
27 --- end
28
29 This is the last page of the survey.
30
31 ```{r}
32 sd_close()
33 ```
```

```

1 ---
2 survey-settings:
3   mode: preview
4 ---
5
6 ```{r}
7 library(surveydown)
8 ```
9
10 --- welcome
11
12 # Welcome to our survey!
13
14 ```{r}
15 sd_question(
16   type = "mc",
17   id = "penguins",
18   label = "What's your favorite penguin?",
19   option = c(
20     "Adélie" = "adelie",
21     "Chinstrap" = "chinstrap",
22     "Gentoo" = "gentoo"
23   )
24 )
25 ```
26
27 --- end
28
29 This is the last page of the survey.
30
31 ```{r}
32 sd_close()
33 ```

```

YAML header (survey settings)

`mode: preview` saves responses to a local file while you build

Load `surveydown` package

```

1 ---
2 survey-settings:
3   mode: preview
4 ---
5
6 ```{r}
7 library(surveydown)
8 ```
9
10 --- welcome
11
12 # Welcome to our survey!
13
14 ```{r}
15 sd_question(
16   type = "mc",
17   id   = "penguins",
18   label = "What's your favorite penguin?",
19   option = c(
20     "Adélie"   = "adelie",
21     "Chinstrap" = "chinstrap",
22     "Gentoo"   = "gentoo"
23   )
24 )
25 ```
26
27 --- end
28
29 This is the last page of the survey.
30
31 ```{r}
32 sd_close()
33 ```

```

Start each page with
 --- and a page id:

```

1 --- page_id
2
3 Page content

```

No closing tag needed:
 the next --- starts the next page

```

1 ---
2 survey-settings:
3   mode: preview
4 ---
5
6 ```{r}
7 library(surveydown)
8 ```
9
10 --- welcome
11
12 # Welcome to our survey!
13
14 ```{r}
15 sd_question(
16   type = "mc",
17   id   = "penguins",
18   label = "What's your favorite penguin?",
19   option = c(
20     "Adélie"   = "adelie",
21     "Chinstrap" = "chinstrap",
22     "Gentoo"   = "gentoo"
23   )
24 )
25 ```
26
27 --- end
28
29 This is the last page of the survey.
30
31 ```{r}
32 sd_close()
33 ```

```

Use markdown for page content

Use `sd_question()` in a code chunk for survey questions

A **Next** button is added to every page automatically

```

1 ---
2 survey-settings:
3   mode: preview
4 ---
5
6 ```{r}
7 library(surveydown)
8 ```
9
10 --- welcome
11
12 # Welcome to our survey!
13
14 ```{r}
15 sd_question(
16   type = "mc",
17   id   = "penguins",
18   label = "What's your favorite penguin?",
19   option = c(
20     "Adélie"   = "adelie",
21     "Chinstrap" = "chinstrap",
22     "Gentoo"   = "gentoo"
23   )
24 )
25 ```
26
27 --- end
28
29 This is the last page of the survey.
30
31 ```{r}
32 sd_close()
33 ```

```

Use `sd_close()` on the last page for an "Exit Survey" button

app.R

```
1 library(surveydown)
2
3 # Connects to database
4 db <- sd_db_connect()
5
6 # Main UI
7 ui <- sd_ui()
8
9 server <- function(input, output, session) {
10   # Main server
11   sd_server(db)
12 }
13
14 shiny::shinyApp(
15   ui = ui,
16   server = server
17 )
```

```
1 library(surveydown)
2
3 # Connects to database
4 db <- sd_db_connect()
5
6 # Main UI
7 ui <- sd_ui()
8
9 server <- function(input, output, session) {
10   # Main server
11   sd_server(db)
12 }
13
14 shiny::shinyApp(
15   ui = ui,
16   server = server
17 )
```

Load package

Connect to database

Make standard UI

Run surveydown server

Your turn

05:00

Step 1: Open the `demo-question-types` folder in Positron
(File › Open Folder...)

Step 2: Open `app.R` and click the “Run App” arrow in the top-left (near where the “preview” button usually is for Quarto docs)

Step 3: Click through the survey, then look at `survey.qmd` to see how each question was made

Start a new survey from a **template**

In the R console, run:

```
1 surveydown::sd_create_survey()
```

Optional: Specify a **path** and / or a **template**:

```
1 surveydown::sd_create_survey(  
2   template = "question_types",  
3   path = "path/to/folder"  
4 )
```

Warning

Every surveydown survey should be in its own separate project folder.

Week 6: *Intro to surveydown*

1. What is surveydown?
2. **Survey basics**
3. Conditional logic
4. Storing & fetching data
5. Reactivity

YAML header

All survey settings go in the YAML header at the top of `survey.qmd`

```
1  ---
2  theme-settings:
3    theme: default
4    barposition: top
5  survey-settings:
6    mode: preview
7    show-previous: no
8    all-required: no
9    required: []
10 ---
```

No header at all? Everything uses the defaults.

Full list of settings on the [Survey Settings](#) docs page

Survey mode

Set under `survey-settings` in the YAML header:

```
1 survey-settings:  
2   mode: preview
```

mode	Where responses are saved
preview	preview_data.csv in your survey folder
local	local_data.csv in your survey folder
database (default)	Your database (see part 4)

Use `preview` while you build, switch to `database` before you launch

Change the survey theme

Pick a different [bootstrap theme](#) with the `theme` key:

```
1 theme-settings:  
2   theme: united
```

Make a custom theme with a `custom.scss` file:

```
1 theme-settings:  
2   theme: [united, custom.scss]
```

Progress bar

Modify the survey progress bar with the `barcolor` and `barposition` keys:

Change to any color with `barcolor`:

```
1 theme-settings:  
2   barcolor: "#768692"
```

Change position: `top`, `bottom`, or `none`

```
1 theme-settings:  
2   barposition: bottom
```

Inserting pages

Start each page with `---` and a page id:

```
1 --- page_id
2
3 Page content here
```

Next buttons are added automatically.

Add a **Previous** button to every page in the YAML:

```
1 survey-settings:
2   show-previous: yes
```

Or control one page's buttons with `sd_nav()` (e.g. change the label):

```
1 ```{r}
2 sd_nav(label_next = "Continue")
3 ```
```

Inserting questions

Insert questions using the `sd_question()` function, like this:

Code

```
1 sd_question(  
2     type = 'mc',  
3     id = 'fruit',  
4     label = "1. Do you like fruit?",  
5     option = c(  
6         'Yes!' = 'yes',  
7         'Kind of' = 'kind_of',  
8         'No :(' = 'no'  
9     )  
10 )
```

Output

1. Do you like fruit?

- ☐ Yes!
- ☐ Kind of
- ☐ No :(

*

Format is `"Label" = "value"`

surveydown supports lots of question types

Some common types you may want to use:

Type	Description
<code>mc</code>	Multiple choice question (single choice)
<code>mc_multiple</code>	Multiple choice question (multiple choices)
<code>mc_buttons</code>	Multiple choice question (large buttons)
<code>select</code>	Drop down menu (choose one)
<code>text</code>	Open text, single row
<code>textarea</code>	Open text, block
<code>numeric</code>	Numbers only
<code>matrix</code>	Several questions with the same options (e.g. agree / disagree)

Or define questions in a `questions.yml` file

In `questions.yml`:

```
1 fruit:
2   type: mc
3   label: 1. Do you like fruit?
4   options:
5     Yes!: yes
6     Kind of: kind_of
7     No :(: no
```

In `survey.qmd`:

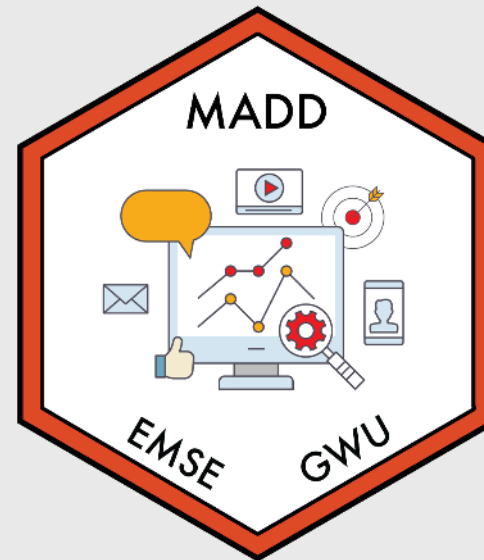
```
1 ```{r}
2 sd_question("fruit")
3 ```
```

See the `demo-questions-yml` folder

Embed images with html

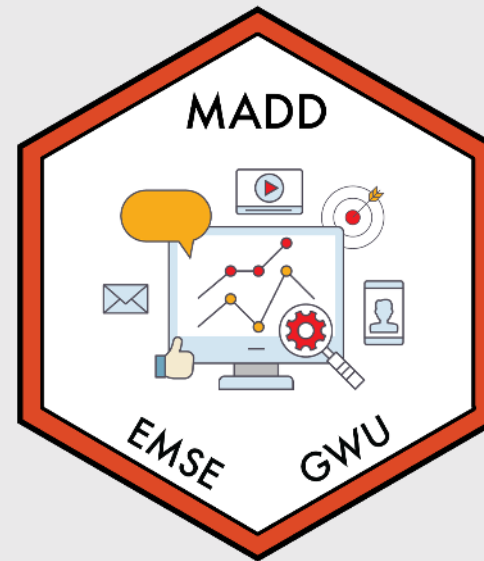
I recommend just writing html code, like this

```
1 <center>  
2   
3 </center>
```



Embed images with markdown

```
1 {fig-align="center" width="250"}
```



Try out the surveydown **studio app**

```
sdstudio::launch()
```

Your turn

10:00

- Make a new survey with `surveydown::sd_create_survey()` and open its folder in Positron.
- Pick a topic for your survey (a food, an animal, a sports team...whatever).
- Draft a survey about that topic in `survey.qmd`. Include the following:
 - Page 1: A welcome message in large font ("Welcome to a survey about [topic]") and an image about the topic (find an image somewhere).
 - Pages 2 to N: Add different questions about the topic using several different question types.
- Try out different themes to change the look and feel.

Week 6: *Intro to surveydown*

1. What is surveydown?
2. Survey basics
3. Conditional logic
4. Storing & fetching data
5. Reactivity

Three conditional logic functions

All go in the `server()` function in `app.R`:

Function	What it does
<code>sd_show_if()</code>	Show a question or page if a condition is <code>TRUE</code>
<code>sd_skip_if()</code>	Skip forward to a page if a condition is <code>TRUE</code>
<code>sd_stop_if()</code>	Stop navigation (with a message) if a condition is <code>TRUE</code>

Conditional Showing

Conditional question showing

Which is your favorite type of penguin?

- ☐ Adélie
- ☐ Chinstrap
- ☐ Gentoo
- ☐ Other

Next

Conditional question showing

Define both *conditional* and *target* questions in `survey.qmd`

```
1 # Conditional question
2 sd_question(
3   type = "mc",
4   id = "penguins",
5   label = "Which is your favorite type of penguin?",
6   option = c(
7     "Adélie" = "adelie",
8     "Chinstrap" = "chinstrap",
9     "Gentoo" = "gentoo",
10    "Other" = "other"
11  )
12 )
13
14 # Target question
15 sd_question(
16   type = "text",
17   id = "penguins_other",
18   label = "Please specify the other penguin type:"
19 )
```

Conditional question showing

Define condition with `sd_show_if()` in `app.R`

```
1 server <- function(input, output, session) {  
2   sd_show_if(  
3     sd_value("penguins") == "other" ~ "penguins_other"  
4   )  
5  
6   # ...other server code...  
7 }
```

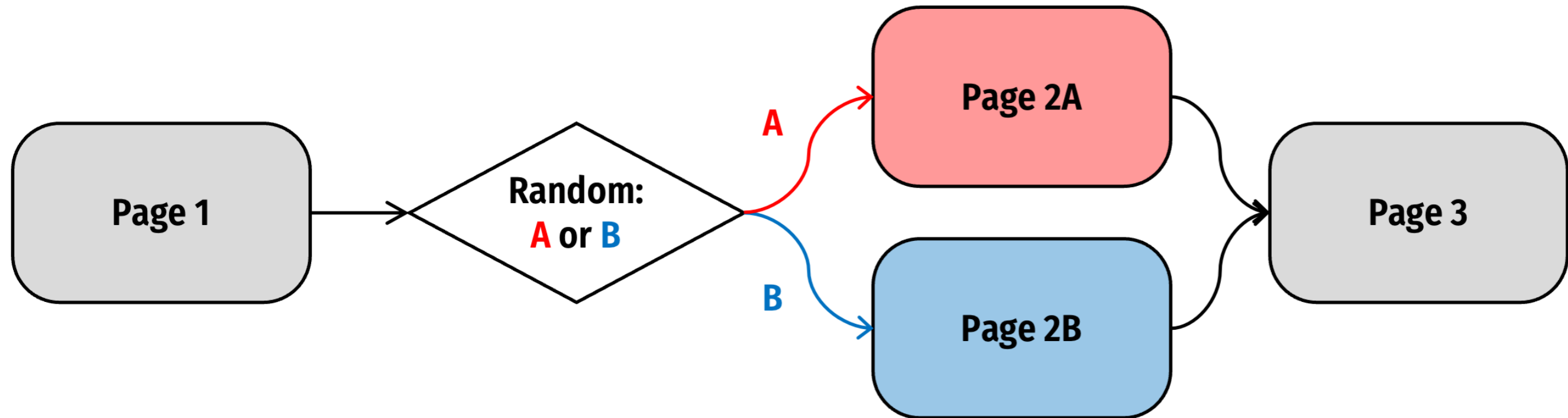
1. The structure of the condition is always:

```
1 <condition> ~ "target_question_id"
```

2. Use `sd_value("id")` to get the value of a question

Conditional Page Showing

Example: Randomly show one of two page versions



First, define all pages in your `survey.qmd` file:

```
1 --- page1
2
3 This is page 1
4
5 --- page2a
6
7 This is page 2A
8
9 --- page2b
10
11 This is page 2B
12
13 --- page3
14
15 This is page 3
16
17 ```{r}
18 sd_close()
19 ```
```

In `app.R`:

```
1 server <- function(input, output, session) {
2   # Generate random condition
3   rand_val <- sample(c('A', 'B'), 1)
4
5   # Store the condition value
6   sd_store_value(rand_val)
7
8   # Use sd_show_if to show target pages
9   sd_show_if(
10     rand_val == 'A' ~ 'page2a',
11     rand_val == 'B' ~ 'page2b'
12   )
13
14   sd_server()
15 }
```

`sd_store_value()` saves `rand_val` in your response data, so you know which version each respondent saw

Conditional Skipping

Example: **Participant screen out**

Example: **Participant screen out**

Do you own a vehicle?

☐ Yes

☐ No

Next

In `survey.qmd`: Target screenout question

```
1 sd_question(  
2   type = 'mc',  
3   id = 'vehicle_ownership',  
4   label = "Do you own a vehicle?",  
5   option = c(  
6     'Yes' = 'yes',  
7     'No' = 'no'  
8   )  
9 )
```

In `app.R`:

```
1 server <- function(input, output, session)  
2   sd_skip_if(  
3     sd_value("vehicle_ownership") == "no" ~  
4   )  
5  
6   # ...other server code...  
7 }
```

Target screenout page

```
1 --- screenout  
2  
3 Sorry, but you are not qual
```

For all three functions, there are lots of
conditions to choose from

See the `demo-conditional-showing` and
`demo-conditional-skipping` folders

Your turn

15:00

- Open the survey you were editing from your last practice.
- Add conditional logic content:
 - Add one simple multiple choice question about your survey topic and another that will only show depending on a specific choice in the first question.
 - Add a target page that respondents should be sent to based on their choices in another question in an earlier page (e.g., a multiple choice question).

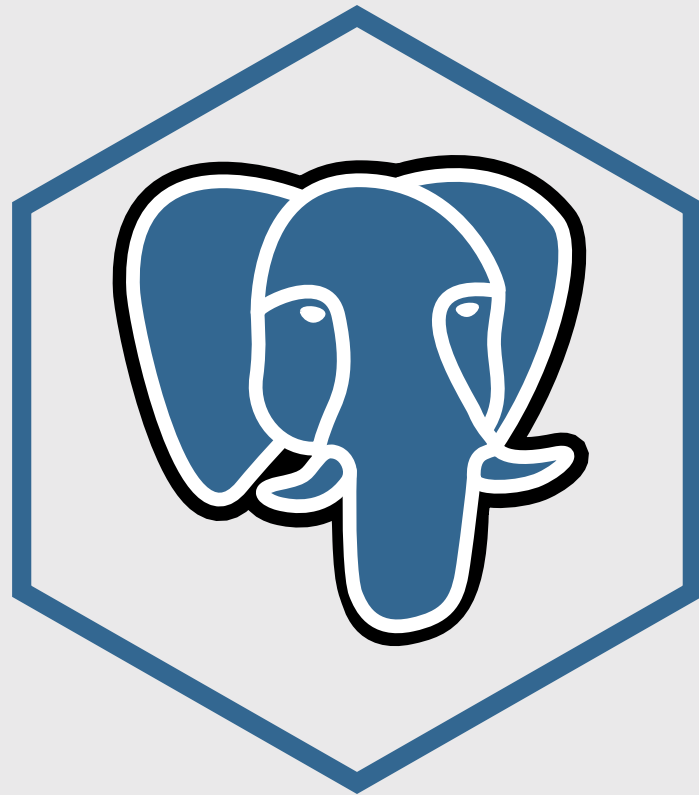
Break

05 : 00

Week 6: *Intro to surveydown*

1. What is surveydown?
2. Survey basics
3. Conditional logic
4. Storing & fetching data
5. Reactivity

Data is stored in any PostgreSQL database



PostgreSQL

We use [Supabase](#) as a free, open-source option



Supabase

Store data in [Supabase](#)

Steps to connect a database via Supabase:

1. Create a Supabase account
2. Create a Supabase project
3. Copy your credentials

Full details on the [Storing Data docs page](#)

Creating a project

Create a new project

Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization

Project name

Project name

Database Password

Type in a strong password

This is the password to your postgres database, so it must be strong and hard to guess. [Generate a password](#)

Region

 West US (North California)

Select the region closest to your users for the best performance.

SECURITY OPTIONS >

Cancel

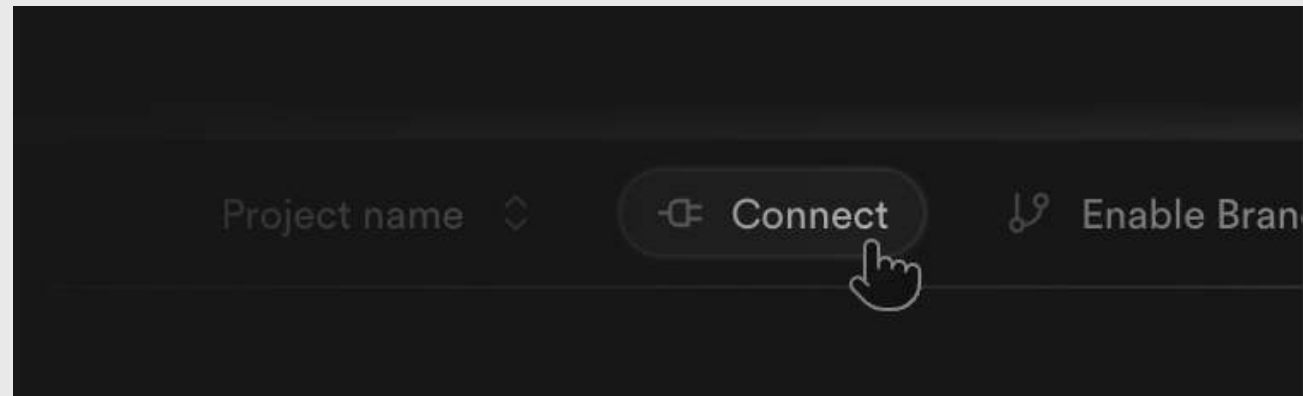
You can rename your project later

Create new project

- Choose a project name (this is your "database")
- Each database can have multiple tables
- Choose a strong password

Getting your Supabase credentials

Click the "connect" button in your project



Choose "Direct", then find the "Transaction pooler" section

Transaction pooler

Ideal for stateless applications like serverless functions where each interaction with Postgres is brief and isolated.

```
postgresql://postgres.axzkymwaxcdkbushwxb:[YOUR-PASSWORD]@
```

Does not support PREPARE statements

View parameters

host: aws-0-us-east-1.pooler.supabase.com

port: 6543

database: postgres

user: postgres.axzkymwaxcdkbushwxb

pool_mode: transaction

For security reasons, your database password is never shown.

Store your database credentials

In your R console, run:

```
1 surveydown::sd_db_config()
```

Credentials are stored in a `.env` file in your survey folder.

Never share or commit this file — it has your password.

```
> sd_db_config()
```

— Database Configuration Setup —

Press Enter to keep current value shown in brackets

Host [aws-0-us-east-1.pooler.supabase.com]:

Port [6543]:

Database name [postgres]:

User [postgres.axzkymwaxcdkbushwxb]:

Password [****]:

Table name [mytable1]:

GSS encryption mode [disable]:

✓ Database configuration updated

— Current database configuration: —

SD_HOST=aws-0-us-east-1.pooler.supabase.com

SD_PORT=6543

SD_DBNAME=postgres

SD_USER=postgres.axzkymwaxcdkbushwxb

SD_TABLE=mytable1

SD_PASSWORD=****

SD_GSSENCMODE=disable

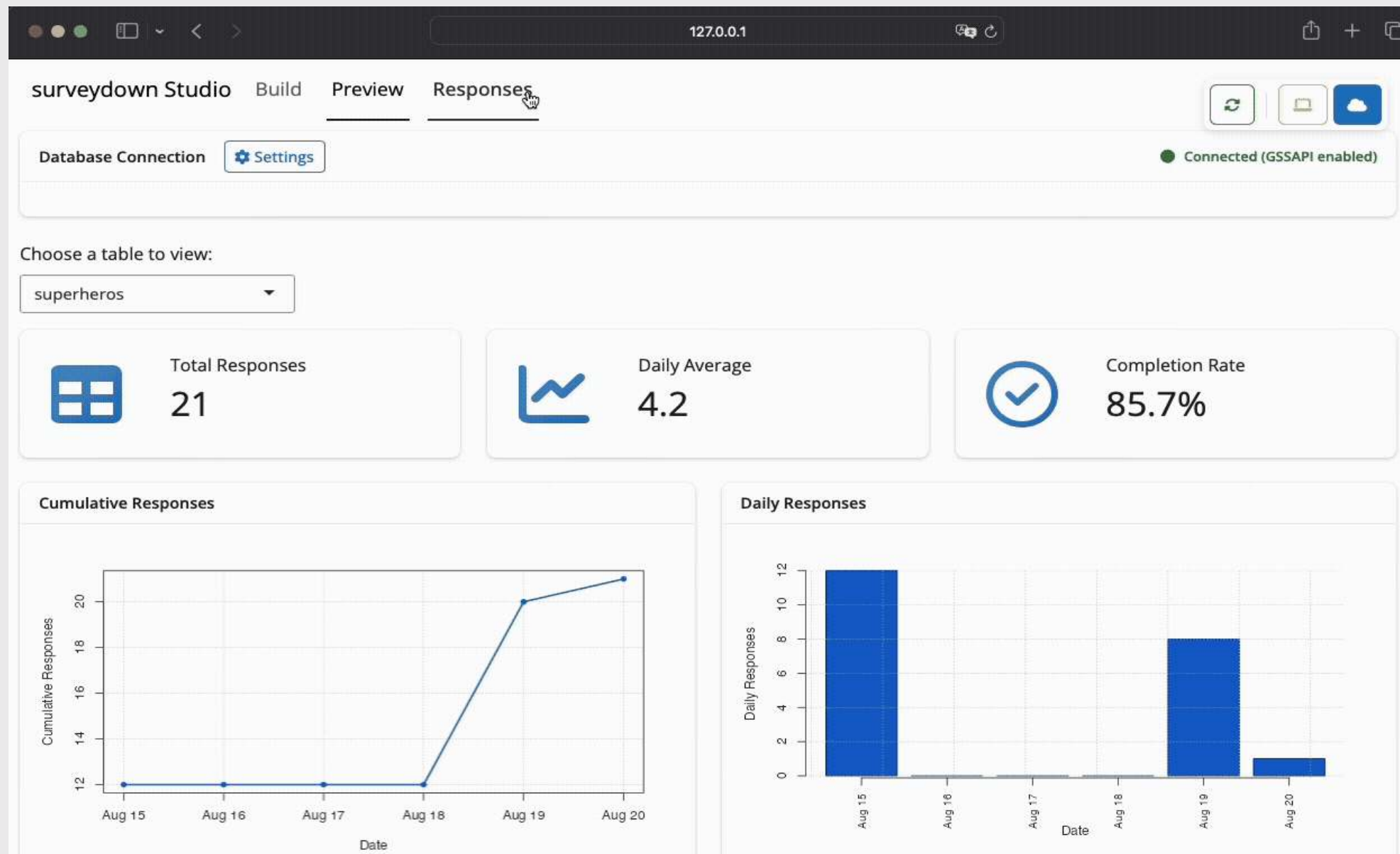
app.R

```
1 library(surveydown)
2
3 # Connects to database
4 db <- sd_db_connect()
5
6 # Main UI
7 ui <- sd_ui()
8
9 server <- function(input, output, session) {
10   # Main server
11   sd_server(db)
12 }
13
14 shiny::shinyApp(
15   ui = ui,
16   server = server
17 )
```

`sd_db_connect()` uses the `.env` file to make the database connection.

Then set `mode: database` in `survey.qmd` (or delete the `mode` line — it's the default)

Use `sdstudio::launch()` to locally view data



Your turn

10:00

- Create a Supabase account and database.
- Run `surveydown::sd_db_config()` in your console to store your Supabase credentials.
- Change `mode: preview` to `mode: database` in your `survey.qmd`.
- Run your survey locally, answer questions to generate data.
- View your response data with `sdstudio::launch()`

Week 6: *Intro to surveydown*

1. What is surveydown?
2. Survey basics
3. Conditional logic
4. Storing & fetching data
5. **Reactivity**

Read the [chapter](#) in Mastering Shiny to understand reactivity in general

Read the surveydown [reactivity page](#) for reactivity in surveydown

Reactivity is about changing
something based on respondent input

What's your name?

Hello J!

What's your name?

Hello Jo!

What's your name?

Hello Joe!

Same example in surveydown

In `survey.qmd`:

```
1 ```{r}
2 sd_question(
3   type = "text",
4   id   = "name",
5   label = "What is your name?"
6 )
7
8 textOutput("greeting")
9 ```
```

In `app.R`:

```
1 server <- function(input, output, session) {
2   output$greeting <- renderText({
3     paste0("Hello ", sd_value("name"), "!")
4   })
5
6   sd_server()
7 }
```

Simplified with `sd_output()` to display question values

In `survey.qmd`:

```
1 ```{r}
2 sd_question(
3   type = "text",
4   id   = "name",
5   label = "What is your name?"
6 )
7 ```
8
9 Hello, `r sd_output("name", type = "value")`!
```

In `app.R`:

```
1 server <- function(input, output, session) {
2   sd_server()
3 }
```

Understanding `sd_output()`

Question in `survey.qmd`:

```
1 sd_question(  
2   type = "mc",  
3   id = "penguins",  
4   label = "What's your favorite penguin?",  
5   option = c(  
6     "Adélie" = "adelie",  
7     "Chinstrap" = "chinstrap",  
8     "Gentoo" = "gentoo"  
9   )  
10 )
```

Display a question *value*:

```
1 sd_output("penguins", type = "value")
```

Example: `adelie`

Display a question *option label*:

```
1 sd_output("penguins", type = "label_option")
```

Example: `Adélie`

Understanding `sd_output()`

Question in `survey.qmd`:

```
1 sd_question(  
2   type = "mc",  
3   id = "penguins",  
4   label = "What's your favorite penguin?",  
5   option = c(  
6     "Adélie" = "adelie",  
7     "Chinstrap" = "chinstrap",  
8     "Gentoo" = "gentoo"  
9   )  
10 )
```

Display a *question label*:

```
1 sd_output("penguins", type = "label_question")
```

Example:

What's your favorite penguin?

Your turn

07:00

In your survey, display a question *value* somewhere with:

```
1 `r sd_output("id", type = "value")`
```

In your survey, display a question *option label* somewhere with:

```
1 `r sd_output("id", type = "label_option")`
```

Blog post on conjoint in surveydown

Next team deliverable:
Pilot Survey. (Due 10/13)